Smoothed Particle Hydrodynamics (SPH) Simulation

Course: IG3D
Name: Arafat ISLAM **ID:** 21501835
Teacher: Professor Dr. Kiwon Um, Telecom Paris

1. Introduction

In this lab, I implemented a basic 2D SPH fluid simulator in the provided code. The rendering and window management parts were already given, so I mainly completed the missing solver functions in `main.cpp`. The goal of this lab was to simulate a liquid block using particle-based fluid equations and obtain a result close to the example shown in the lab manual.

I implemented the main parts asked in the lab: neighbor search, density computation, pressure, body force, viscosity and particle update. After that, I tested the simulation and adjusted it to get a stable fluid motion.

2. Neighbor Search

The first important step was to find neighboring particles efficiently. Instead of checking all particles against all others, I used the background grid already provided in the code.

Each grid cell stores the indices of particles inside it:

$$_pidxInGrid[\text{idx1d}(i, j)]$$

Then, for each particle, I only checked the current cell and the surrounding cells. This made the SPH computation much faster.

The implemented code is:

```
_pidxInGrid.clear();
_pidxInGrid.resize(resX()*resY());

for(tIndex i=0; i<particleCount(); ++i) {
  const int x = static_cast<int>(
    clamp(std::floor(_pos[i].x), 0, resX()-1));
  const int y = static_cast<int>(
    clamp(std::floor(_pos[i].y), 0, resY()-1));
  _pidxInGrid[idx1d(x, y)].push_back(i);
}
```

3. Density Computation

After building the grid, I computed the density of each particle using the standard SPH density equation:

$$\rho_i = \sum_j m_j W_{ij}$$

Here, W_{ij} is the cubic spline kernel and m_j is the particle mass. In the code, I looped over neighboring cells and summed the contributions:

```
_d[i] = 0;
const Vec2f rij = _pos[i] - _pos[*it];
_d[i] += _m0*_kernel.w(rij);
```

When this part worked correctly, the particle color also changed because the blue channel depends on ρ_i/ρ_0.

4. Pressure

After computing density, I computed pressure with the equation of state:

$$p_i = k\left[\left(\frac{\rho_i}{\rho_0}\right)^{\gamma} - 1\right]$$

In my implementation, I used:

```
return std::max(k*(std::pow(d/d0, gamma) - 1e0), 0e0);
```

I kept the pressure non-negative because without this clamp the simulation became unstable and particles spread unrealistically. Then I computed the pressure force with the symmetric SPH formulation:

$$\mathbf{a}_i^{pressure} = -\sum_j m_j \left(\frac{p_i}{\rho_i^2} + \frac{p_j}{\rho_j^2}\right) \nabla W_{ij}$$

The implemented code is:

```
_acc[i] -= _m0*(_p[i]/square(_d[i]) +
  _p[*it]/square(_d[*it])) *
  _kernel.grad_w(rij, len);
```

This term is important because it keeps the liquid from compressing too much and makes it spread naturally.

5. Body Force and Viscosity

For the body force, I only added gravity:

$$\mathbf{a}_i^{body} = \mathbf{g}$$

The code is:

```
for(tIndex i=0; i<particleCount(); ++i)
  _acc[i] += _g;
```

Then I added viscosity with the SPH viscosity formula:

$$\mathbf{f}_i^{viscosity} = 2\nu m_i \sum_j \frac{m_j}{\rho_j} \mathbf{u}_{ij} \frac{\mathbf{x}_{ij} \cdot \nabla W_{ij}}{\mathbf{x}_{ij} \cdot \mathbf{x}_{ij} + 0.01h^2}$$

In the code, I accumulated the acceleration form:

```
const Vec2f uij = _vel[i] - _vel[*it];
const Vec2f gradWij = _kernel.grad_w(xij, len);
const Real coeff =
  xij.dotProduct(gradWij) /
  (xij.dotProduct(xij) + 0.01*square(_h));

_acc[i] += 2e0*_nu*_m0/_d[*it]*uij*coeff;
```

This viscosity term helped smooth the fluid motion and reduced noisy particle movement.

6. Time Integration

After computing all accelerations, I updated velocity and position using the Symplectic Euler method:

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \Delta t\, \mathbf{a}_i(t)$$

$$\mathbf{x}_i(t+\Delta t) = \mathbf{x}_i(t) + \Delta t\, \mathbf{v}_i(t+\Delta t)$$

The code is:

```
_vel[i] += _dt*_acc[i];
_pos[i] += _dt*_vel[i];
```

After this update, the provided collision handling kept the particles inside the simulation box.

7. Final Result

For my final simulation, I used:

- $\Delta t = 0.001$
- `gSolver.initScene(48, 32, 16, 16);`
- one solver step per rendered frame

The simulation starts with a block of particles in the lower-left area of the domain. Then the fluid falls due to gravity, spreads along the floor, and later creates a splash near the right wall. My final result is close to the expected behavior shown in the lab manual.

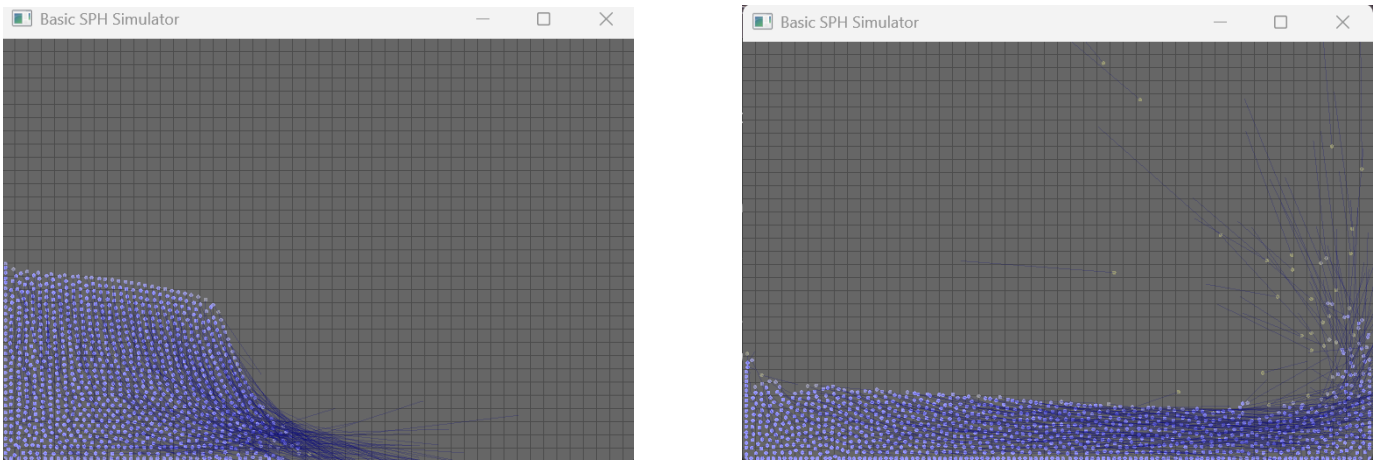

Figure 1: First two frames of my SPH simulation. On the left, the fluid block starts to fall and deform. On the right, the particles spread along the bottom and move toward the right wall.

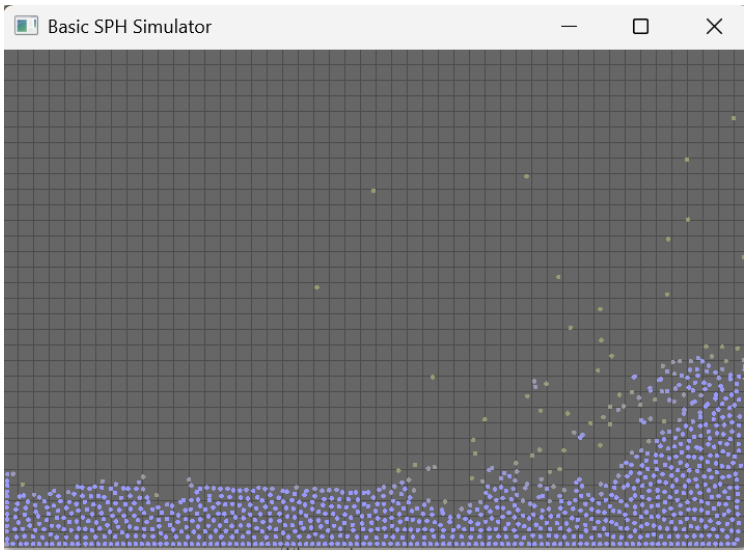

Figure 2: Later simulation frame. The fluid is mostly settled near the bottom and a splash appears near the right wall.

8. Conclusion

In this lab, I implemented the main components of a basic 2D SPH fluid simulator. I completed the neighbor search, density computation, pressure equation, pressure force, gravity, viscosity and Symplectic Euler integration.

The final animation shows the expected liquid behavior: the fluid mass falls, deforms, spreads and interacts with the container boundaries. The most important parts for getting a stable result were the grid-based neighbor search and the pressure computation.